

Lösung (HTB{un0bfu5c4t3d_5tr1ng5})

Dokumentation

Name: Lukas Schmidt

Matrikelnummer: 1498639

Studiengang: FB03 - B.Sc. Informatik

Aufgabennummer: 1

Aufgabenstellung

In dieser Aufgabe ist das einzige Material welches bereitgestellt ist ein Bild (mysterious.jpg), in welchem eine andere, passwortgeschützte Datei zu finden sein soll. Das Ziel ist es, die Datei aus dem Bild zu extrahieren, den Passwortschutz zu hacken und in den Inhalten eine Flag vom Format "HTB{s0me_t3xt}" zu finden. Falls die Datei auf eine weitere Weise verschlüsselt oder unlesbar sein sollte, ist es klug vor der Suche nach der Flag den Inhalt lesbar zu machen.



Vorgehensweise

Datei-Extraktion

Der erste Schritt ist es, herauszufinden auf welche Weise die Datei eingebettet ist. Da es sich um ein Bild handelt, in welches die Datei eingebettet wurde, ist nach kurzer Online-Suche die erste Vermutung, dass "Steganographie" verwendet wurde.

ChatGPT's erste Empfehlung für ein Steganographie-Tool für Linux-Systeme ist "Steghide" ([GitHub](#)), welches mit dem folgenden Befehl auf unser Ubuntu-WSL installiert wird:

```
sudo apt-get install steghide
```

Um nun herauszufinden ob das Tool für dieses Problem kompatibel ist, versuchen wir ersteinmal die Datei aus dem Bild zu extrahieren. Dies tun wir mit folgendem Befehl:

```
steghide extract -sf mysterious.jpg
```

Die darauf folgende Ausgabe ist diese:

```
lukas@Lukas:~$ steghide extract -sf mysterious.jpg
Enter passphrase:
```

Das Tool scheint also zu funktionieren, allerdings müssen wir nun einen Weg finden den Passwortschutz zu umgehen. Die erste Idee hierfür ist ein Brute-Force-Ansatz, welcher eine gegebene Wörterliste an häufig verwendeten Passwörtern automatisiert prüft. Diese Idee scheint nach Blick in die Hilfestellung der korrekte Weg nach vorne zu sein.

Passwort knacken

Das Tool welches ChatGPT für diesen Angriff empfiehlt ist "StegCracker", ein Tool was spezifisch für die Extraktion passwortverschlüsselter, steganographisch eingebetteter Dateien konzipiert wurde.

Allerdings finden wir kurze Zeit später heraus, dass ein neues Tool namens "StegSeek" ([GitHub](#)) um einiges schneller in der selben Aufgabe ist und steigen somit auf dieses Tool um.

Wir installieren StegSeek mit folgendem Befehl:

```
sudo apt-get install stegseek
```

Als Wörterliste verwenden wir die "10-million-password-list-top-1000000.txt" von "SecLists" ([GitHub](#)). Zu guter Letzt beginnen wir den Brute-Force-Angriff mit diesem Befehl:

```
lukas@Lukas:~$ stegseek mysterious.jpg ./SecLists/Passwords/Common-
Credentials/10-million-password-list-top-1000000.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "bloodymary"
[i] Original filename: "pass".
[i] Extracting to "mysterious.jpg.out".
```

Der Angriff war erfolgreich und wir erhalten das Passwort ("bloodymary"), sowie den originalen Dateinamen ("pass") und die extrahierte Datei, welche unter "mysterious.jpg.out" gespeichert wurde.

Finden der Flag

Ein schneller check mithilfe des Befehls `file mysterious.jpg.out` gibt uns folgende Informationen:

```
mysterious.jpg.out: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),  
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,  
BuildID[sha1]=3008217772cc2426c643d69b80a96c715490dd91, for GNU/Linux 4.4.0,  
not stripped
```

Nach kurzer Recherche mithilfe von ChatGPT erkennen wir, dass es sich hierbei um ein Executable handelt. Da wir uns in einem CTF-Szenario befinden und uns somit nicht um die Sicherheit des Programmes sorgen müssen, geben wir uns über den Befehl `chmod 777 mysterious.jpg.out` vollständige Rechte zum Ausführen der Datei, welche uns folgenden Output darbietet:

```
Welcome to the SPOOKIEST party of the year.  
Before we let you in, you'll need to give us the password:
```

Wir benötigen also erneut ein Passwort zum ausführen des Executables. Anstatt jedoch erneut zu einem Brute-Force-Angriff zu greifen, versuchen wir uns zunächst einen Überblick über den Inhalt der Datei zu beschaffen. Über den Befehl `strings mysterious.jpg.out` geben wir uns den Inhalt der Datei in unserem CLI aus und nach ein wenig Suchen finden wir dies:

```
Before we let you in, you'll need to give us the password:  
s3cr3t_p455_f0r_gh05t5_4nd_gh0ul5
```

Es scheint als wäre dies das Passwort, welches zum Ausführen der Datei benötigt wird. Lass es uns also erneut versuchen, dieses mal geben wir jedoch "s3cr3t_p455_f0r_gh05t5_4nd_gh0ul5" als Passwort an.

```
lukas@Lukas:~$ ./mysterious.jpg.out  
Welcome to the SPOOKIEST party of the year.  
Before we let you in, you'll need to give us the password:  
s3cr3t_p455_f0r_gh05t5_4nd_gh0ul5  
Welcome inside!  
HTB{un0bfu5c4t3d_5tr1ng5}
```

Das Programm nimmt das Passwort an und druckt die Flag (HTB{un0bfu5c4t3d_5tr1ng5}) nach welcher wir gesucht haben, womit wir das CTF-Problem gelöst haben.

Flagge

```
HTB{un0bfu5c4t3d_5tr1ng5}
```

Probleme

///

Optional: Fazit/Feedback zur Aufgabe

Diese Aufgabe dient meiner Meinung nach als eine perfekte Einführung in das Thema "Embedded Files", spezifischer den Nutzen von Steganographie, wie man eine eingebettete Datei extrahiert und eventuelle Passwortverschlüsselungen brechen kann.